

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

CO1 AT1 - Application-Based Question Answers

CO1_AT1 – Application-Based Answers (Q1 – Q5)

Course: Design and Analysis of Algorithms (CSA06) **CO1:** Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

Q1. Application: Online Video Streaming Platform

Question

A video streaming platform processes user data using an algorithm with recurrence $T(n) = 2T(n/2) + n$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze how this affects scalability as users increase. Interpret efficiency using asymptotic notation.

Identifying Parameters

- Recurrence: $T(n) = aT(n/b) + f(n) \rightarrow a = 2, b = 2, f(n) = n$
- Watershed function: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

Case Selection and Result

$f(n) = n$ and $n^{\log_b a} = n$ are asymptotically equal, so $f(n) = \Theta(n^{\log_b a})$ — this is Case 2 of the Master Theorem:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)$$

Analysis — Scalability

- $\Theta(n \log n)$ grows only slightly faster than linear, unlike $\Theta(n^2)$, which quadruples when n doubles.
- The split is always balanced ($n/2, n/2$), so best, average, and worst case all remain $\Theta(n \log n)$ — no adversarial input can degrade performance.

Interpretation: This is the same efficient growth class as Merge Sort, so the platform's algorithm remains practical even as the user base scales into the millions.

Q2. Application: Social Media Feed Processing

Question

A social media platform processes posts using a recursive algorithm $T(n) = T(n-1) + n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive the final complexity. Analyze how performance degrades with large inputs. Interpret the result using asymptotic notation.

Step-by-Step Expansion

$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = T(n-3) + (n-2) + (n-1) + n = \dots = T(n-k) + [(n-k+1) + \dots + n]$$

Stopping at $k = n - 1$ ($T(1) = c$):

$$T(n) = c + (2 + 3 + \dots + n) = c + [n(n+1)/2 - 1]$$

$$T(n) = \Theta(n^2)$$

Verification by Induction

Assume $T(k) \leq ck^2$ for $k < n$. Then $T(n) = T(n-1) + n \leq c(n-1)^2 + n = cn^2 - 2cn + c + n \leq cn^2$ for large enough c, n — confirming $\Theta(n^2)$.

Analysis — Performance Degradation

Posts (n)	Approx. operations (n ²)
1,000	1,000,000
10,000	100,000,000

A 10× growth in posts causes a 100× growth in work — unlike Q1's algorithm, this one does not scale to large user bases and should be redesigned toward a linear or $n \log n$ approach.

Q3. Application: E-Commerce Recommendation System

Question

An e-commerce system uses two algorithms with complexities $O(n \log n)$ and $O(n^2)$. Analyze both using asymptotic notation. Compare their performance for large datasets. Explain growth rate differences and justify which is more efficient. Provide a conclusion based on scalability.

Comparative Growth Analysis

n (products)	$n \log n$	n^2
1,000	~9,966	1,000,000
10,000	~132,877	100,000,000
100,000	~1,660,964	10,000,000,000

Explanation of Growth Rate Difference

- $O(n \log n)$ adds only a slowly-growing logarithmic factor on top of linear growth — doubling n roughly doubles the work (plus a small constant more).
- $O(n^2)$ grows quadratically — doubling n roughly quadruples the work, and the gap between the two functions widens dramatically as n increases (as the table shows).

Justification and Conclusion

The $O(n \log n)$ algorithm is significantly more efficient for large product catalogues, since its operation count stays orders of magnitude below the $O(n^2)$ algorithm's as n grows. For an e-commerce platform with a large and growing catalogue, the $O(n \log n)$ recommendation algorithm is the scalable choice; the $O(n^2)$ algorithm would become a performance bottleneck well before catalogue size reaches typical production scale.

Q4. Application: Cloud Storage Processing System

Question

A cloud system processes files using $T(n) = 3T(n/2) + n$. Apply the Master Theorem to find the time complexity. Identify parameters and justify the case selection. Analyze how this impacts performance for large storage systems. Interpret efficiency using asymptotic notation.

Identifying Parameters

- $a = 3, b = 2, f(n) = n$
- Watershed function: $n^{(\log_b a)} = n^{(\log_2 3)} \approx n^{1.585}$

Case Selection and Result

Comparing $f(n) = n$ with $n^{(\log_b a)} \approx n^{1.585}$: $f(n)$ grows polynomially slower ($n = O(n^{(1.585 - \epsilon)})$ for some $\epsilon > 0$), so this is Case 1 of the Master Theorem:

$$T(n) = \Theta(n^{(\log_b a)}) = \Theta(n^{1.585})$$

Analysis — Impact on Large Storage Systems

- $\Theta(n^{1.585})$ lies between linear and quadratic — worse than the $\Theta(n \log n)$ seen in Q1, but far better than $\Theta(n^2)$.
- The recursive splitting into 3 subproblems per level dominates the total cost (most work happens at the leaves of the recursion tree), so the number of subproblems ($a = 3$) is the primary driver of cost, not the combine step $f(n)$.

Interpretation: As storage volume grows, processing time grows faster than $n \log n$ but is still sub-quadratic; the system remains usable for large file volumes but is noticeably less scalable than a balanced 2-way divide-and-conquer design.

Q5. Application: Financial Analytics System

Question

A financial system uses an algorithm with recurrence $T(n) = 2T(n/2) + n \log n$. Apply the Master Theorem and determine the complexity. Compare $f(n)$ with $n^{\log_b a}$ and justify the correct case. Analyze performance for large datasets. Interpret efficiency clearly.

Identifying Parameters

- $a = 2, b = 2, f(n) = n \log n$
- Watershed function: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

Comparing $f(n)$ with $n^{\log_b a}$

$f(n) = n \log n$ grows polynomially faster than $n^{\log_b a} = n$ (by a logarithmic factor), but not by a full polynomial factor n^ϵ . This means the standard polynomial gap required for Case 3 does not strictly hold, so this recurrence falls into the extended form of Case 2 ($f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with $k = 1$):

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n \log^2 n)$$

Analysis — Performance for Large Datasets

- $\Theta(n \log^2 n)$ is only marginally worse than the $\Theta(n \log n)$ seen in Q1 — the extra log-factor comes directly from the $\log n$ combine cost at every recursive level.
- For large financial datasets (e.g., millions of transaction records), this remains highly scalable compared to polynomial complexities like $\Theta(n^{1.585})$ or $\Theta(n^2)$.

Interpretation / Conclusion

The financial analytics algorithm is efficient and close to the ideal $\Theta(n \log n)$ divide-and-conquer performance; the small additional $\log n$ factor from the combine step has negligible practical impact even as transaction volumes scale to large sizes.